🛡️ ShieldAI — IDS/IPS : capture en C

Benny — 10 octobre 2026. Tout ce qu'on a vu ensemble, en langage simple, avec les décisions prises.

1. L'idée en 5 phrases

1. 📡 **C écoute** le réseau (un thread par interface).
2. 🗂️ **C range** les paquets en flux : une fiche par conversation.
3. 📬 Quand une fiche est finie, C la dépose dans une **boîte aux lettres partagée** (un ring en mmap, un par thread).
4. 🐍 **Python vide les boîtes à tour de rôle** (en appelant des fonctions C) et analyse.
5. 🚫 Une **liste d'IP à ignorer** est consultée par C avant de créer une fiche.

2. Décisions prises ✅

- **Capture en C**, un thread par interface, socket AF_PACKET lié à l'interface (un seul socket pour toutes les interfaces = un seul thread qui reçoit tout, donc goulot ; filtrer après coup = travail perdu).
- **Chaque thread** a sa propre flow table et son propre ring mmap. Rien n'est partagé entre threads de capture.
- **Flow table** : ton dict C (collisions chaînées) qui donne un **numéro de nœud**, plus un tableau de nœuds relié en **file doublement chaînée par indices**. Un flux mis à jour passe à la fin de la file ; l'expiration se fait par la tête (le plus ancien).
- **Ring** : mmap nommé (shm_open), **taille en octets**, compteurs w (écriture) et r (lecture) qui ne sont jamais remis à zéro, position = compteur modulo taille. On garde le mmap même en mono-process : on pourra séparer la capture dans un démon plus tard.
- **Python lit via des fonctions C** (ring_open, ring_read) : toute la logique de positions reste en C.
- **Détection de scan** : une table par thread, IP source vers un petit ensemble de ports (portset de 32 ports). Alerte si plus de ~20 ports distincts dans la fenêtre de temps.
- **Scan multi-interface** : un **thread agrégateur en C** reçoit de petits événements (un par nouveau flux) de chaque thread de capture. Au début, Python peut aussi agréger en lisant tous les rings.
- **Set d'IP à ignorer** : table à adressage ouvert avec cases atomiques, lecture **sans lock**, deux instances (deux sets). Ajout par lot depuis Python avec pointeur + taille. Mutex uniquement entre écrivains.
- **Détection en plusieurs process** : possible, avec la règle un ring = un seul lecteur.

3. Les threads C appelés depuis Python

Les threads créés par pthread **survivent** à la fin de la fonction qui les lance, tant que leurs données sont sur le tas (calloc), pas sur la pile.

```
typedef struct {
    pthread_t th[MAX_T];
    int n;
    _Atomic int stop;
    ring_t *rings[MAX_T];
} engine_t;

engine_t *engine_start(const char *iface, int n, size_t ring_bytes); //
revient tout de suite
void engine_stop(engine_t *e);   // stop=1, pthread_join de chaque thread,
free
```

- Le `pthread_t` est stocké dans la struct : le join se fait dans engine_stop.
- **GIL** : aucun problème. Les threads C ne touchent jamais à Python. Et ctypes.CDLL relâche le GIL pendant l'appel.
- Un segfault en C **tue tout le processus Python**.

```
gcc -O2 -shared -fPIC -pthread engine.c -o libengine.so
```

```
import ctypes as C
lib = C.CDLL('./libengine.so')
lib.engine_start.argtypes = [C.c_char_p, C.c_int, C.c_size_t]
lib.engine_start.restype  = C.c_void_p   # SINON l'adresse 64 bits est
tronquée
lib.engine_stop.argtypes  = [C.c_void_p]
h = lib.engine_start(b'eth0', 4, 64*1024*1024)
# ... dans un try/finally ou atexit :
lib.engine_stop(h)
```

Un pointeur en Python est un simple **int** (type déclaré : c_void_p). On ne le déréférence jamais en Python : on le repasse à C.

4. Le ring (la boîte aux lettres)

🧒 Un tapis roulant de sushi : le chef (C) pose à w, toi (Python) tu manges à r. Si le tapis est plein, le chef attend (on drop et on compte).

```
typedef struct {
    uint64_t magic, size, rec_size;   // infos sur le ring
    char pad0[40];                    // complète la ligne de cache
    _Alignas(64) _Atomic uint64_t w;  // écrit par C
    _Alignas(64) _Atomic uint64_t r;  // écrit par le lecteur
    _Alignas(64) unsigned char data[];
} ring_t;                             // les données commencent à l'offset
192

int ring_push(ring_t *g, const void *src, size_t len) {
    uint64_t w = atomic_load_explicit(&g->w, memory_order_relaxed);
    uint64_t r = atomic_load_explicit(&g->r, memory_order_acquire);
    if (g->size - (w - r) < len) return 0;                 // plein
    size_t pos = w % g->size;
    size_t first = g->size - pos; if (first > len) first = len;
    memcpy(g->data + pos, src, first);                     // fin du buffer
    memcpy(g->data, (const char*)src + first, len - first); // reste au début
    atomic_store_explicit(&g->w, w + len, memory_order_release); // publier
EN DERNIER
    return 1;
}
```

- **Pourquoi des lignes différentes** : le CPU lit la RAM par blocs de 64 octets. Si w et r sont dans le même bloc, les deux cœurs se le disputent (false sharing). `_Alignas(64)` place chaque compteur dans son propre bloc.
- **Taille en octets** : la taille est fixée par ftruncate ; le mmap ne peut pas grossir.
- **Modulo** : correct pour n'importe quelle taille, car des compteurs uint64 en octets ne bouclent jamais en pratique. Le masque (&) n'est utile que pour une puissance de 2.
- **Records coupés** : si les records font tous 32 octets et que la taille est un multiple de 32, aucun record n'est coupé. Sinon le memcpy en deux temps gère le cas.
- **Python** : ring_open(nom) renvoie un handle (int), ring_read(handle, buffer, max) copie des flux dans un tableau numpy et renvoie combien.
- Un ring = un seul écrivain et un seul lecteur. Aucun lock.

5. La flow table

```
dict : clé du flux (src, dst, sport, dport, proto, iface)  ->  numéro i
nodes[i] : données du flux + prev + next (des indices, -1 = rien) + last_seen
```

À chaque paquet : chercher la clé, puis mettre à jour nodes[i] et le passer à la fin de la file. Sinon prendre un nœud libre, le remplir, l'ajouter au dict et à la fin de la file.

- Expiration : tant que la tête est trop vieille, écrire le flux dans le ring, le retirer du dict, rendre le nœud.
- **Indices plutôt que pointeurs** : tableau de taille fixe (mémoire bornée), copiable et sauvegardable tel quel.
- Le dict stocke un numéro, pas les données : déplacer un flux à la fin ne touche qu'à prev/next.
- **Collisions** : chaînage, et toujours comparer la **clé complète** avant de dire trouvé.
- Si le dict renvoie 0 pour absent, l'indice 0 est ambigu : stocker i+1 ou renvoyer -1.
- Le numéro d'interface fait partie de la clé.

6. Détection d'attaques

Un flux n'a qu'un seul port destination. Pour voir un scan, il faut regarder **par IP source**, donc une table séparée de la flow table.

```
typedef struct { uint16_t ports[32]; uint8_t n; } portset_t;

int portset_visit(portset_t *s, uint16_t p) {
    for (int i = 0; i < s->n; i++) if (s->ports[i] == p) return 0; // déjà vu
    if (s->n < 32) s->ports[s->n++] = p;
    return 1;
}
// si s->n >= 20 dans la fenêtre : scan de ports
```

À chaque **nouveau flux** (SYN sans ACK) : chercher l'IP source, portset_visit(dport), tester le seuil. Chaque entrée a un last_seen et est recyclée après un timeout (par exemple 60 s).

Ce qui varie permet de reconnaître l'attaque :

- 🚪 **Les ports** (1 source vers 1 cible) : scan de ports.
- 🏘️ **Les IP cibles** (1 source, 1 port) : balayage réseau (network sweep). Compter les IP destination distinctes par source.
- 👥 **Les IP sources** (plein de sources vers 1 cible) : DDoS. Table par IP cible : nombre de sources distinctes et débit.
- 🔁 **Rien ne varie mais énormément** (1 source, 1 cible, 1 port) : DoS simple, SYN flood (beaucoup de SYN sans suite) ou brute force (SSH, page de login).

Le rythme compte : 50 connexions vers le port 443 en 10 s, c'est un site web normal (1 seul port distinct, donc pas un scan). Attention au NAT : une même IP peut cacher beaucoup de vraies personnes, donc des seuils trop bas donnent des faux positifs.

La **bitmap** (65 536 bits = 1024 uint64, 8 Ko) donne un compte exact de ports distincts : on teste le bit port % 64 du mot port / 64. À garder pour plus tard. Un grand nombre hexa de bits, c'est exactement ça : 1 chiffre hexa = 4 bits, le OU (|) allume un bit et le ET (&) le teste.

7. Plusieurs interfaces

- Sans bind (ou ifindex 0), un socket reçoit toutes les interfaces avec leur sll_ifindex, mais on garde **un thread par interface**. Les gros outils (Suricata, Zeek) travaillent aussi avec des workers par interface ou par queue.
- Un scan sur deux interfaces est rare mais possible : d'où l'**agrégateur**.

```
typedef struct { uint32_t src, dst; uint16_t dport; uint8_t iface, pad;
double ts; } evt_t;
```

Un événement seulement pour un **nouveau flux**, pas à chaque paquet. Un ring par thread de capture ; l'agrégateur les lit à tour de rôle et alimente la table IP source vers portset.

⚠️ **Mode BRIDGE** : un paquet qui traverse le bridge est vu deux fois (entrée et sortie). Filtrer avec `sll_pkttype == PACKET_OUTGOING` pour ne garder que l'entrée, sinon un scan de 100 ports compte 200.

8. Le set d'IP à ignorer

```
#define CAP 4096   // puissance de 2, rempli à 50 % maximum
typedef struct { _Atomic uint32_t slot[CAP]; pthread_mutex_t wlock; uint32_t
count; } ipset_t;

int ipset_has(ipset_t *s, uint32_t ip) {          // lecteurs : AUCUN lock
    uint32_t i = (ip * 2654435761u) >> 20;        // 12 bits : 0..4095
    for (uint32_t n = 0; n < CAP; n++, i = (i + 1) & (CAP - 1)) {
        uint32_t v = atomic_load_explicit(&s->slot[i], memory_order_acquire);
        if (v == 0)  return 0;                    // case vide : absent
        if (v == ip) return 1;
    }
    return 0;
}
```

- Une case passe de 0 à ip en **une seule écriture atomique** : un lecteur voit vide ou complet, jamais à moitié.
- Le **for borné** garantit la fin même si la table est pleine (un while(1) bloquerait le thread de capture et l'IDS deviendrait aveugle).

- **0 = case vide**, donc l'IP 0.0.0.0 ne peut pas être ajoutée.
- **50 %** : en recherche ratée, ~2,5 cases à regarder à 50 %, ~6 à 70 %, ~50 à 90 %, l'infini à 100 %. Table de 16 Ko, donc la place perdue ne coûte presque rien.
- Pas de realloc ni de free sur cette table (les lecteurs seraient en train de lire un endroit qui n'existe plus). Pas de suppression simple (possible avec une valeur tombstone comme 0xFFFFFFFF que les lecteurs sautent).
- **Ajout par lot** : `int ipset_add_many(ipset_t *s, const uint32_t *ips, size_t n)` prend le mutex **une seule fois** et boucle sur une fonction interne sans mutex (le mutex n'est pas récursif : appeler ipset_add dedans bloquerait). Refuser si count dépasse 50 % de CAP. C ne garde pas le pointeur après le return.

```
import ctypes as C, ipaddress
lib.ipset_add_many.argtypes = [C.c_void_p, C.POINTER(C.c_uint32), C.c_size_t]
lib.ipset_add_many.restype  = C.c_int

def add_ips(s, ips):
    arr = (C.c_uint32 * len(ips))(*(int(ipaddress.IPv4Address(i)) for i in
ips))
    return lib.ipset_add_many(s, arr, len(ips))
```

- **Ordre des octets** : int(IPv4Address) est en ordre lisible ; dans un paquet brut l'IP est en ordre réseau. Faire ntohl() côté C à la lecture du paquet.
- Le dict de Benny peut servir de set (clé = IP, valeur bidon), mais il faudrait le rendre sûr pour la lecture concurrente (cases atomiques, pas de resize, nœuds remplis avant d'être accrochés, pas de free). La table ci-dessus est plus simple : choisie.

9. Mini-lexique 📚

- **_Atomic** : opérations indivisibles (pas de valeur à moitié écrite), pas de valeur gardée en registre, pas de réorganisation par le compilateur. Il faut `#include <stdatomic.h>`.
- **Fonctions utiles** : atomic_store (écrire), atomic_load (lire), atomic_fetch_add (compteur partagé). Plus avancées : atomic_exchange, atomic_compare_exchange_strong (CAS, base du lock-free multi-écrivains).
- **Ordres mémoire** : relaxed (juste atomique), acquire (lecture), release (écriture qui publie), seq_cst (le plus strict, défaut).
- **malloc** : n octets, contenu aléatoire. **calloc** : n éléments mis à zéro. **realloc** : change la taille, le bloc peut déménager, utiliser un pointeur temporaire. **aligned_alloc(64, taille)** : adresse alignée, taille multiple de 64. **free** : rendre le bloc (un alloc = un free).
- **sizeof** : la taille dépend du type, donc sizeof(struct A) = sizeof(une instance). Calculé à la compilation, avec du padding.

- **strtok_r** : split de chaîne en C (travailler sur une copie, car il modifie la chaîne). Mieux vaut faire le parsing en Python et passer pointeur + taille.
- **Python : le double étoile** : dans `(C.c_uint32 * n)(*liste)`, le premier * fabrique le type tableau de n uint32 ; le second * renverse la liste en arguments séparés (f(*[1,2,3]) = f(1,2,3)).
- **Hash plus modulo** : position = hash % taille ; les collisions sont inévitables, donc prévoir chaînage ou case suivante.

10. Pièges à ne pas oublier ⚠️

- restype = c_void_p en ctypes, sinon le pointeur est tronqué.
- Jamais de realloc sur une table lue par d'autres threads.
- Toujours borner les boucles de recherche (for, pas while(1)).
- Écrire la donnée d'abord, publier le compteur (release) en dernier.
- Comparer la clé complète dans les chaînes de collision.
- Compter les paquets une seule fois en mode bridge.
- Une IP derrière un NAT peut représenter beaucoup de monde.
- Ne pas appeler une fonction qui prend le mutex depuis une fonction qui l'a déjà pris.

11. Prochaines étapes 🛠️

1. Renvoyer le code du dict C pour relecture (collisions, comparaison de clé, resize).
2. engine_start / engine_stop, avec un thread de capture AF_PACKET par interface.
3. Flow table : tableau de nœuds, free list, file LRU, expiration.
4. Ring : ring_open / ring_push / ring_read, test de bout en bout avec Python.
5. ipset (has, add, add_many) et branchement avant la création d'un flux.
6. Table IP source vers portset, avec seuils et fenêtre de temps.
7. Thread agrégateur et evt_t.
8. Plus tard : balayage réseau, table par IP cible pour le DDoS, bitmap exacte, suppression dans ipset, plusieurs process de détection.

Valeurs à choisir : taille des rings, MAX_FLOWS, durée des timeouts, seuil de ports distincts, taille de la fenêtre.